



栈和队列的高级主题选讲

- 单调栈
- 单调队列
- 求栈元素的极值
- 最大频率栈

数据之法
结构之美
算法之道



zhuyungang@jlu.edu.cn

例（下一个更大元素）：给定一个长度为 n 的数组 a ，找出每个位置的下一个更大元素（即对于位置 i ，找 $a[i]$ 的右边第一个比 $a[i]$ 大的元素），如果不存在，则输出-1。 n 不超过 10^5 。【字节跳动面试题】

例如：

输入数组 a 10 8 6 5 9 12

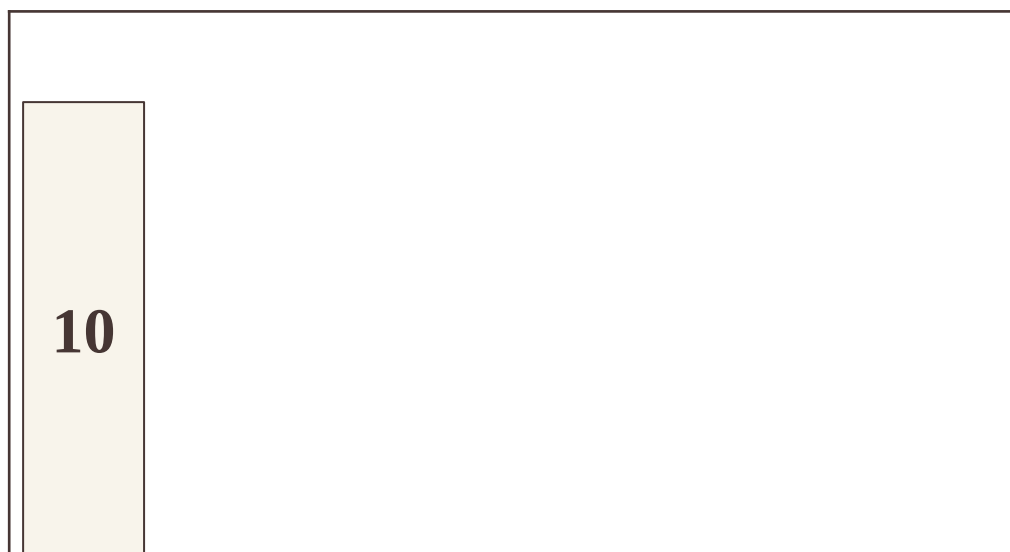
输出数组 $right$ 12 9 9 9 12 -1

$right[i]$ ：元素 $a[i]$ 的下一个更大元素

a 10 8 6 5 9 12



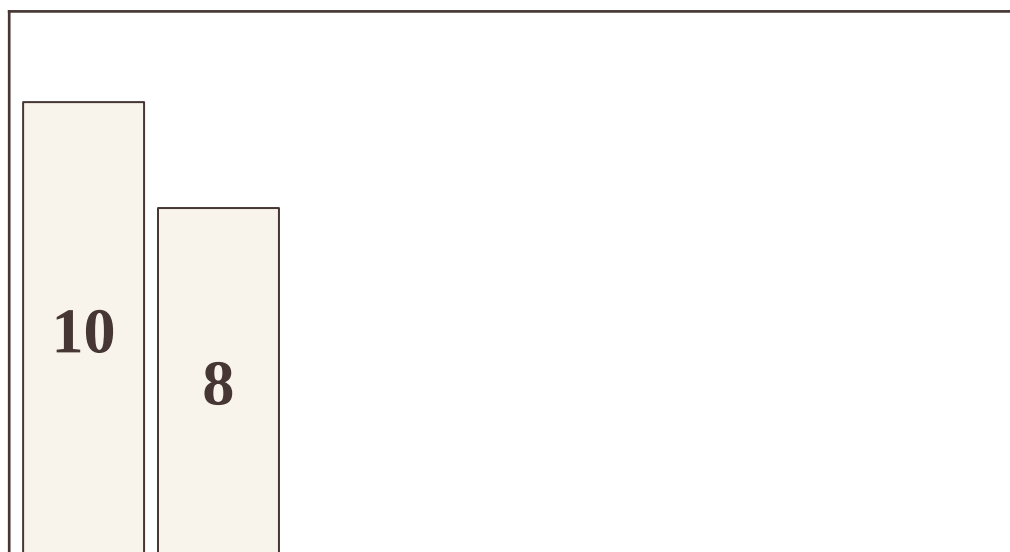
当前元素	栈顶元素	动作
10	空	10入栈



a 10 8 6 5 9 12



当前元素	栈顶元素	动作
8	10	8入栈

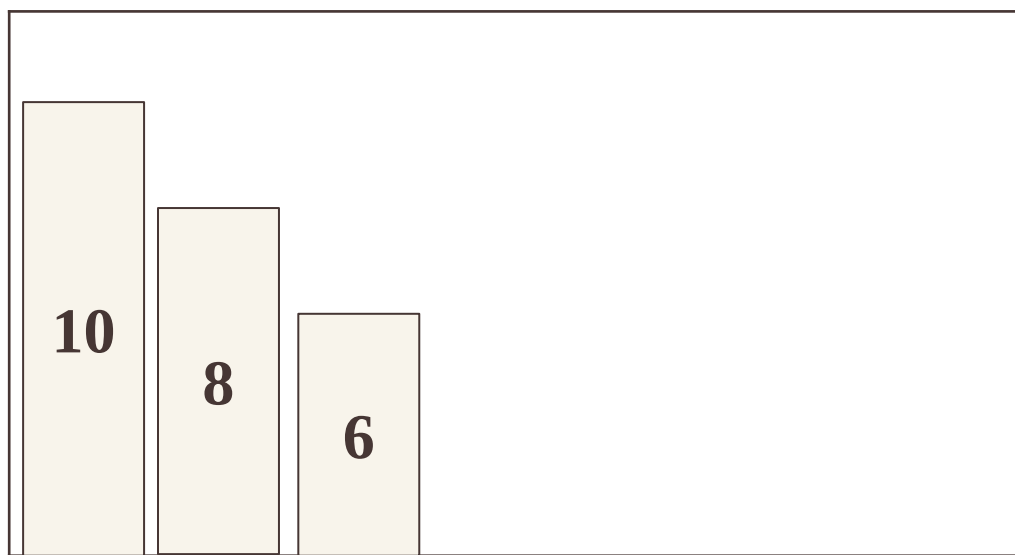


当前元素 $\leq$ 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
6	8	6入栈

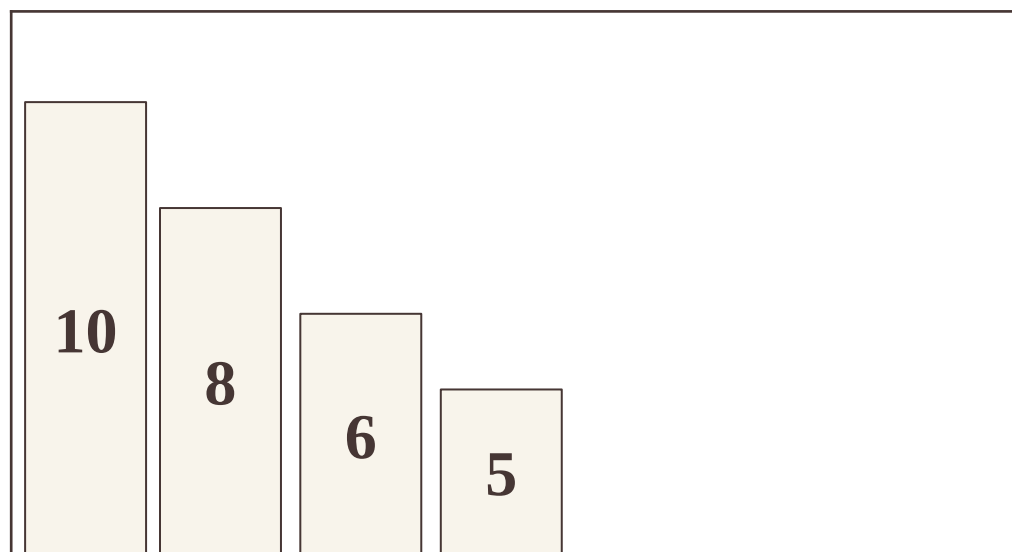


当前元素 $\leq$ 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
5	6	5入栈

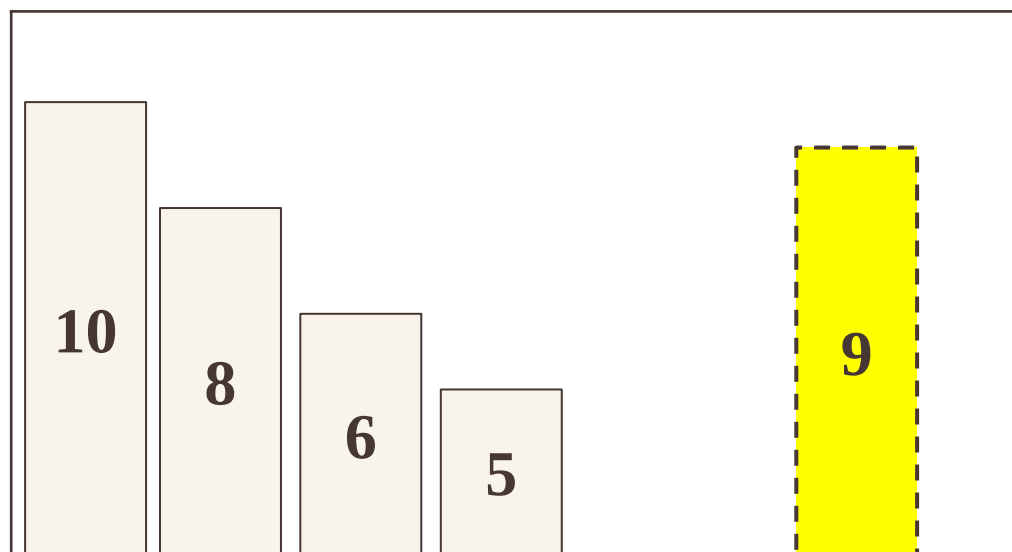


当前元素 $\leq$ 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
9	5	5出栈



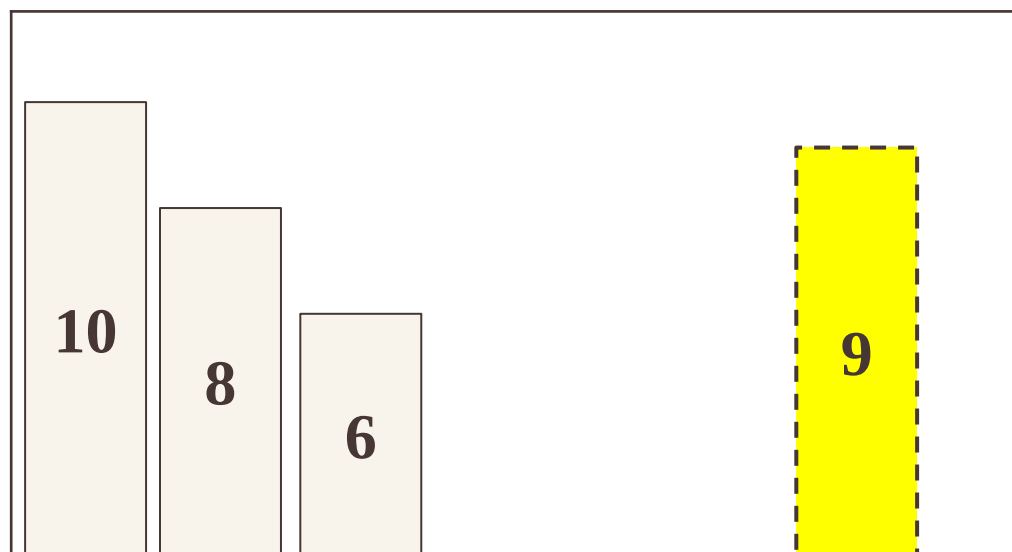
5的下一个更大元素：9
 $\text{right}[3]=9$

当前元素 > 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
9	6	6出栈



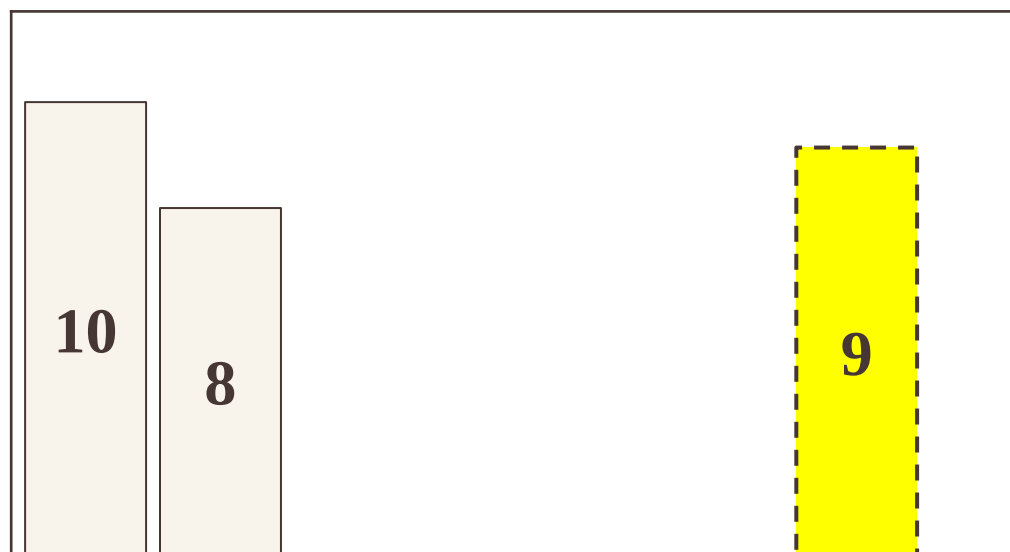
6的下一个更大元素：9
 $\text{right}[2]=9$

当前元素 > 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
9	8	8出栈



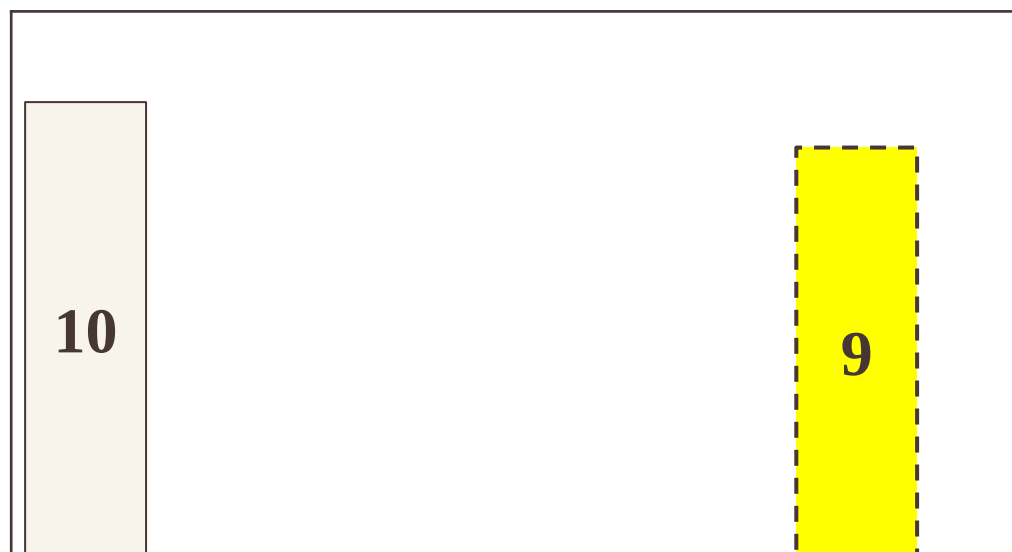
8的下一个更大元素：9
 $\text{right}[1]=9$

当前元素 > 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
9	10	9入栈

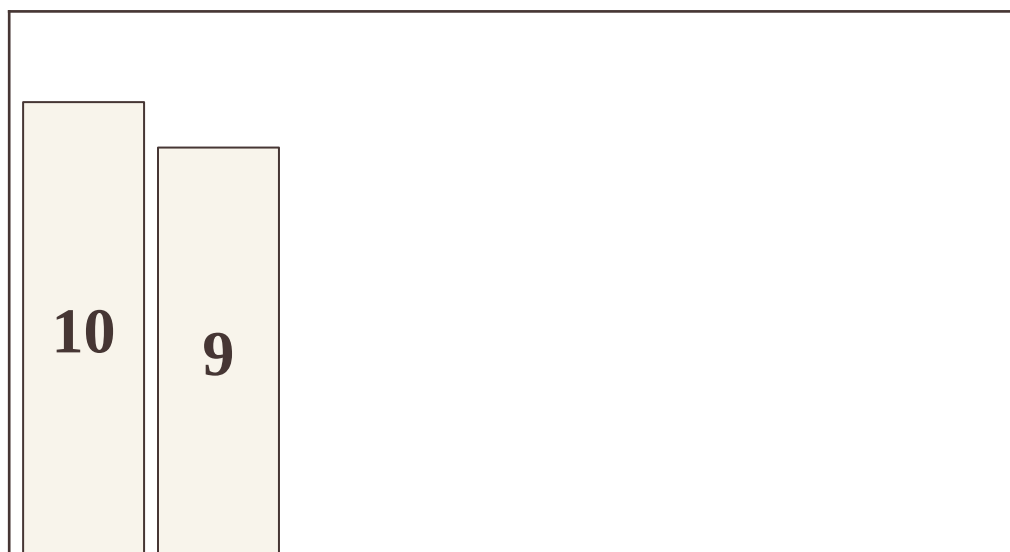


当前元素 $\leq$ 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
9	10	9入栈



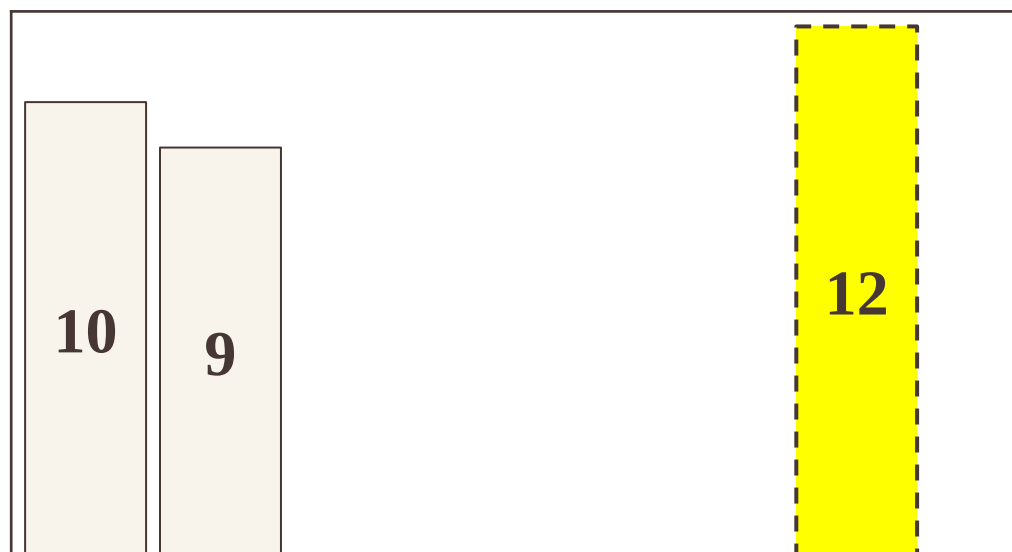
- ✓ 令当前元素为 K
- ✓ $K \leq$ 栈顶：压栈
- ✓ $K >$ 栈顶：弹栈（弹出的元素的下一个更大元素正是 K ）直至 $K \leq$ 栈顶或栈空，然后再把 K 压栈
- ✓ 栈内元素单调递减

当前元素 $\leq$ 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
12	9	9出栈



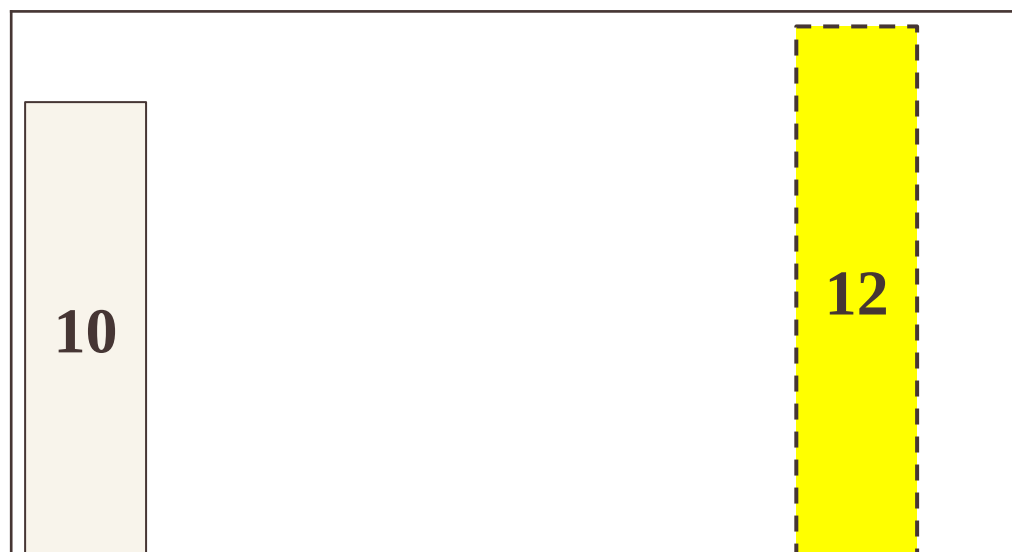
9的下一个更大
元素：12
right[4]=12

当前元素 > 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
12	10	10出栈



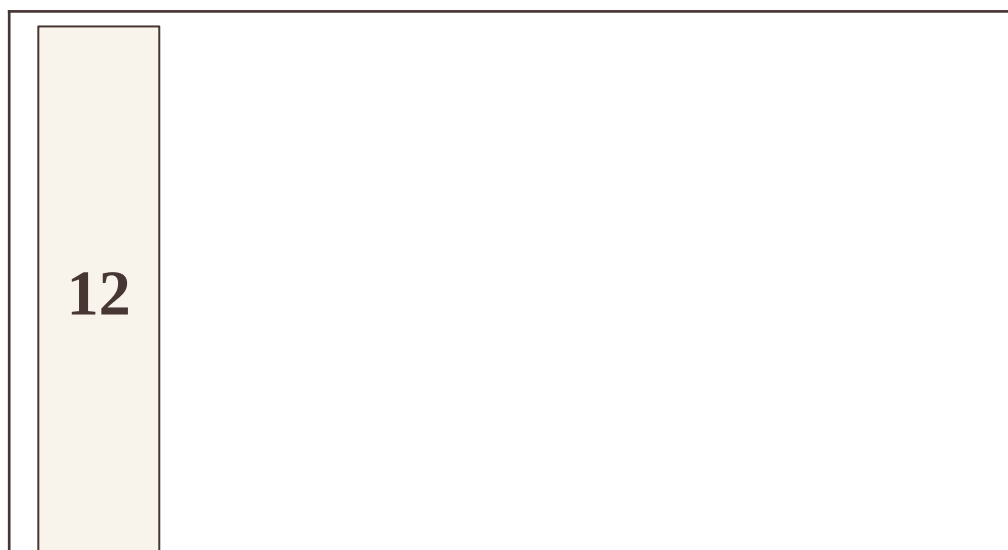
10的下一个更大元素：12
 $\text{right}[0]=12$

当前元素 > 栈顶元素

a 10 8 6 5 9 12



当前元素	栈顶元素	动作
12	空	12入栈



- ✓ 令当前元素为 K
- ✓ $K \leq$ 栈顶：压栈
- ✓ $K >$ 栈顶：弹栈（弹出的元素的下一个更大元素正是 K ）直至 $K \leq$ 栈顶或栈空，然后再把 K 压栈
- ✓ 栈内元素单调递减

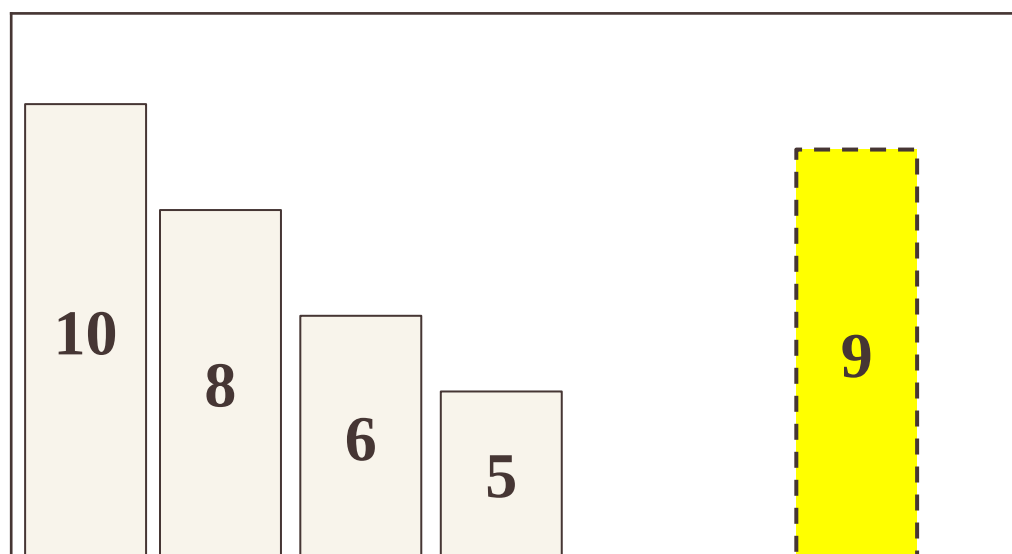
时间复杂度 $O(n)$

a 10 8 6 5 9 12

问题：如何确定出栈元素的下标？



当前元素	栈顶元素	动作
9	5	5出栈



5的下一个更大元素：9
 $\text{right}[3]=9$

技巧：栈中存元素在数组*a*中的下标

实现一个“乞丐版”栈

✓ 定义并初始化栈：

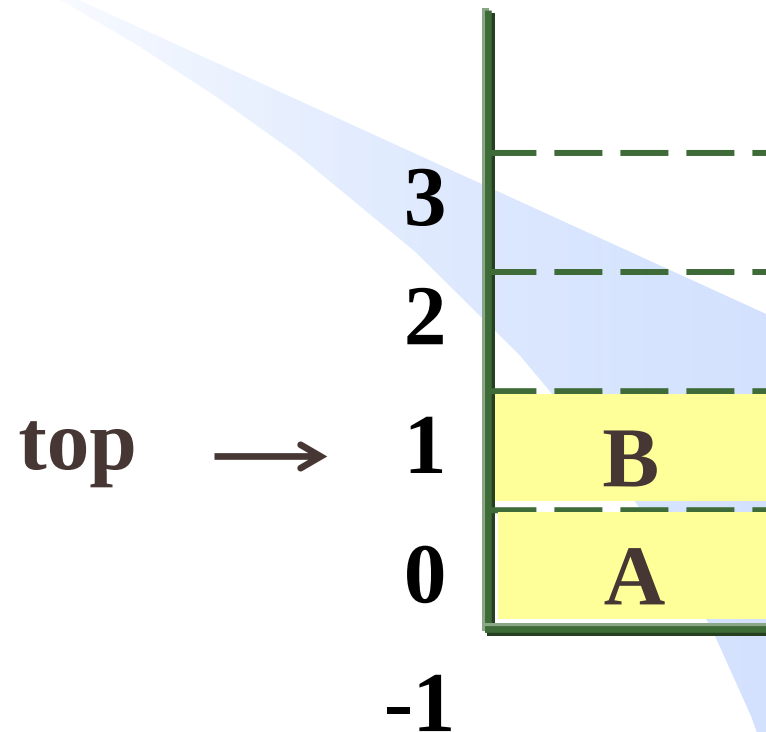
```
const int maxn = 1e5 + 10;
```

```
int stack[maxn], top = -1;
```

✓ 判断栈空： `top == -1`

✓ 压栈 `i`： `stack[++top] = i;`

✓ 弹栈： `return stack[top--];`

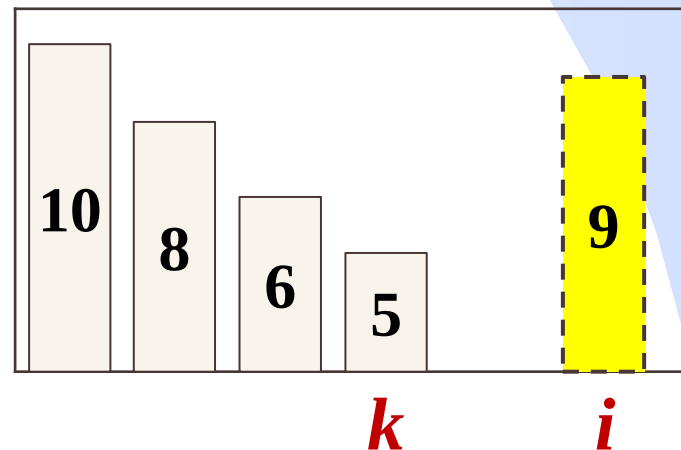


单调（递减）栈

时间复杂度 $O(n)$

```
const int maxn = 1e5 + 10;
void NextGreater(int a[], int right[], int n) {
    for(int i = 0; i < n; i++) right[i] = -1; //初始化
    int stack[maxn], top = -1; //定义并初始化栈
    for(int i = 0; i < n; i++) { //扫描数组a[]中的元素
        while(top >= 0 && a[i] > a[stack[top]]){
            int k = stack[top--]; //弹栈
            right[k] = a[i]; //a[k]的下一个更大元素是a[i]
        }
        stack[++top] = i; //当前元素压栈
    }
}
```

为什么right数组初始化为-1?





例（上一个更大元素）：给定一个长度为 n 的数组 a ，找出每个位置的**上一个**更大元素（即对于位置 i ，找 $a[i]$ 的**左边**第一个比 $a[i]$ 大的元素），如果不存在，则输出 -1 。 n 不超过 10^5 。

【字节跳动面试题】

例如：

输入数组 a 10 8 6 5 9 12

输出数组left -1 10 8 6 10 -1

left $[i]$ ：元素 $a[i]$ 的上一个更大元素

a 倒过来： 12 9 5 6 8 10



```
const int maxn = 1e5 + 10;
void PreviousGreater(int a[], int left[], int n) {
    for(int i = 0; i < n; i++) left[i] = -1;
    int stack[maxn], top = -1;
    for(int i = n-1; i >= 0 ; i--){
        while(top >= 0 && a[i] > a[stack[top]]){
            int k = stack[top--];
            left[k] = a[i];
        }
        stack[++top] = i;
    }
}
```

反向扫描



例（每日温度）：给定每日气温列表，请计算在每一天需要等几天才会有更高的温度。如果气温在这之后都不会升高，请在该位置用 0 来代替。列表长度不超过 10^5 。【字节跳动、华为、阿里、腾讯、谷歌、微软、苹果面试题[LeetCode739](#)】

例：

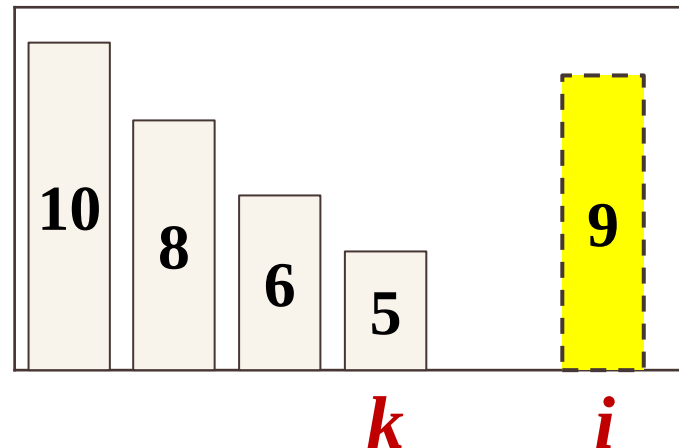
输入：33 34 35 31 29 32 36 33

输出：1 1 4 2 1 1 0 0

```

const int maxn = 1e5 + 10;
void DailyTemperature(int a[], int right[], int n){
    for(int i=0; i<n; i++) right[i]=0;
    int stack[maxn], top=-1;
    for(int i=0; i<n; i++) {
        while(top>=0 && a[i]>a[stack[top]]){
            int k=stack[top--];
            right[k]=i - k; //a[k]的下一个更大元素是a[i]
        }
        stack[++top]=i;
    }
}

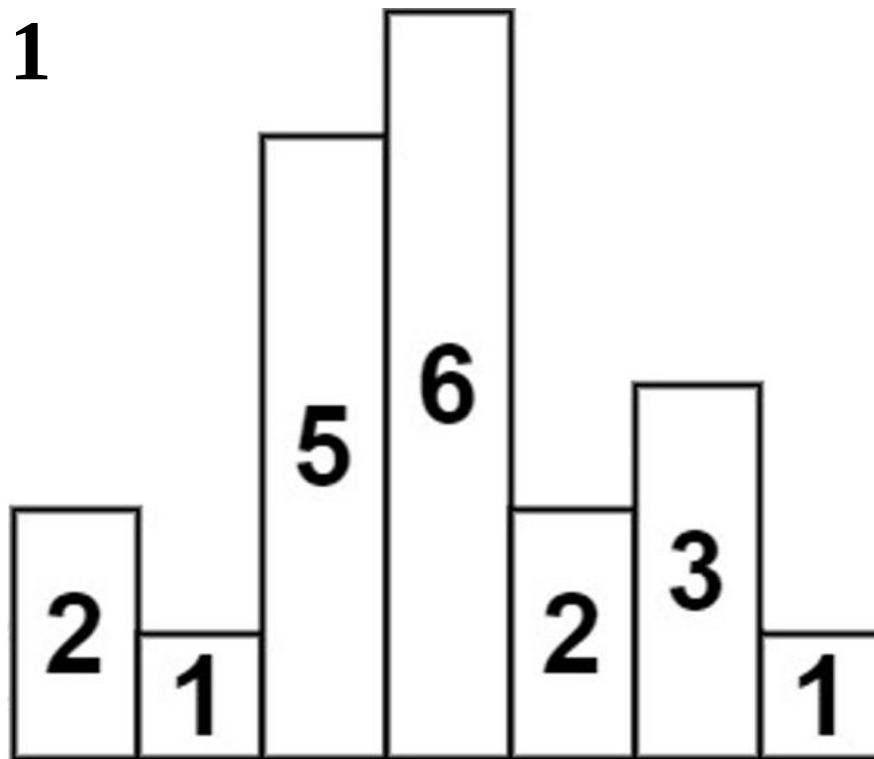
```

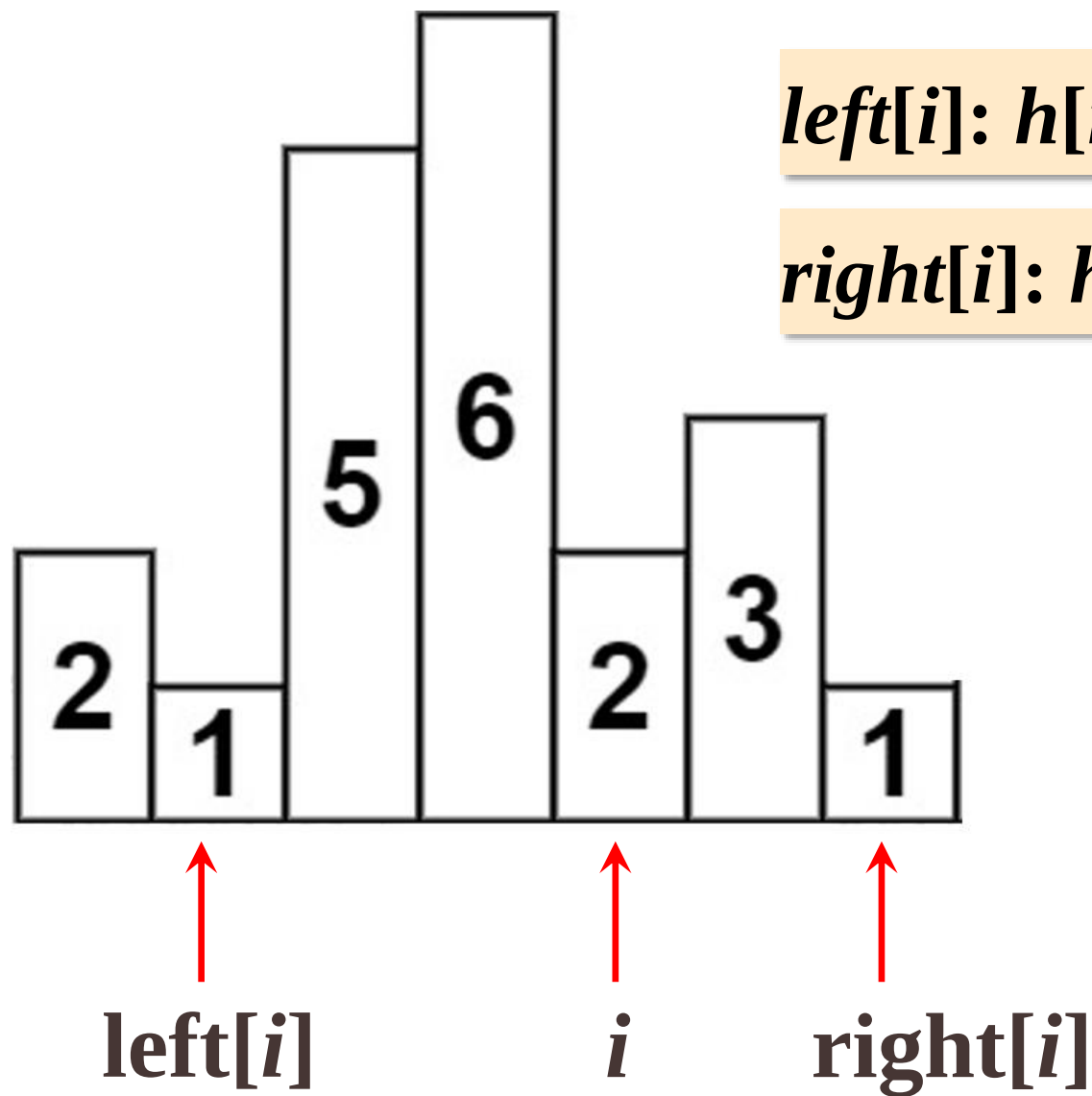


例（柱状图中最大矩形）：给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。求在该柱状图中，能够勾勒出来的矩形的最大面积。【腾讯、华为、字节跳动、微软、谷歌、苹果、百度、小米、网易面试题、CSP认证真题CSP201312-3、LeetCode84】

示例输入：2 1 5 6 2 3 1

示例输出：10





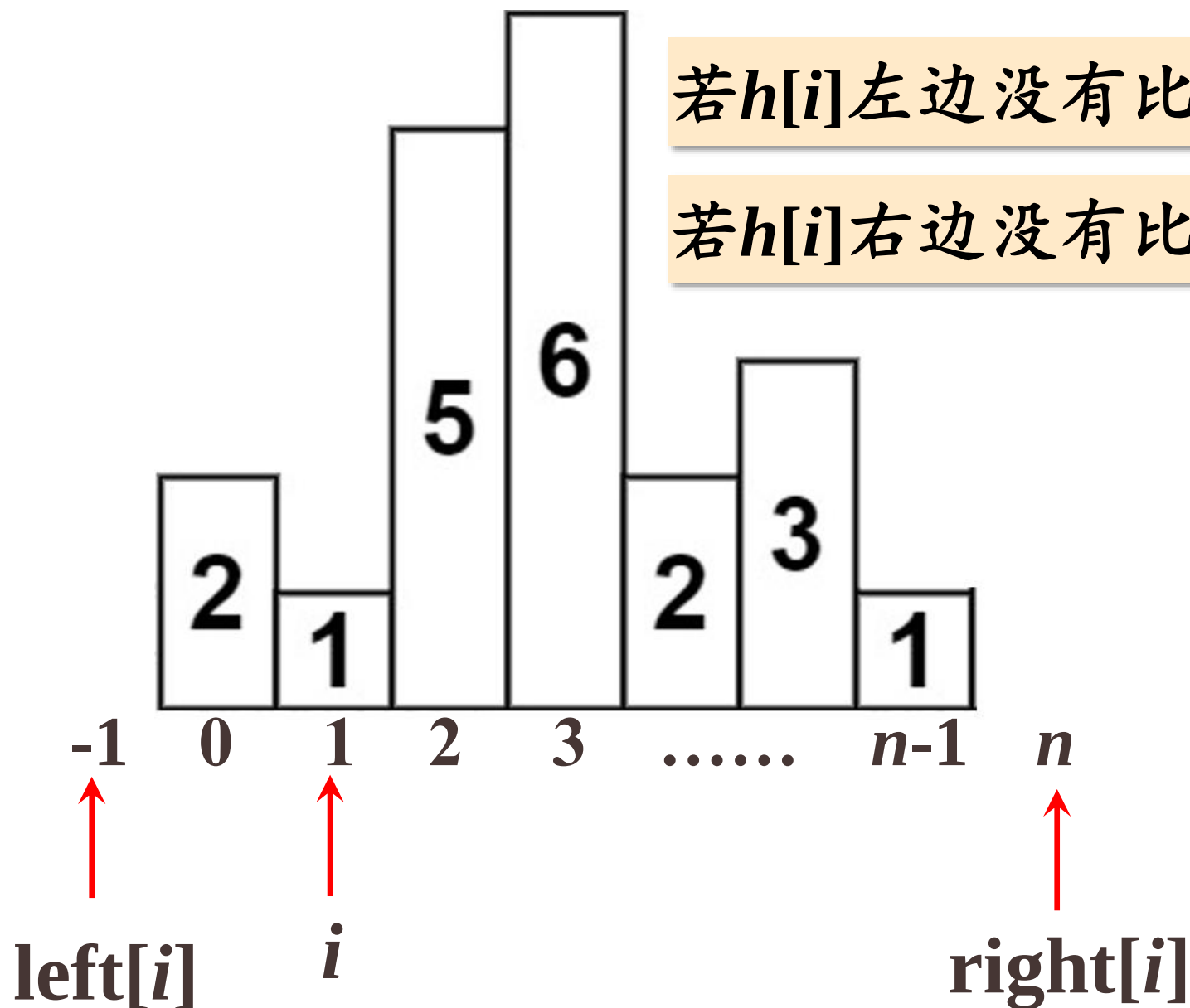
$left[i]$: $h[i]$ 左边第一个比 $h[i]$ 小的位置

$right[i]$: $h[i]$ 右边第一个比 $h[i]$ 小的位置

第 i 号柱子所能勾勒出来的最大矩形面积
 $h[i] * (right[i] - left[i] - 1)$

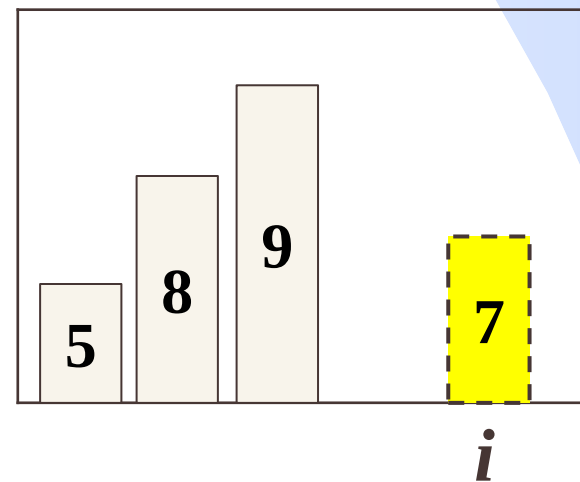
若 $h[i]$ 左边没有比它小的元素，则 $left[i] = -1$

若 $h[i]$ 右边没有比它小的元素，则 $right[i] = n$



维护单调递增栈

```
const int maxn = 1e5 + 10;  
void NextSmaller(int h[], int right[], int n) {  
    //计算right[i], 即h[i]右边第1个比h[i]小的元素位置  
    for(int i=0; i<n; i++) right[i]=n;  
    int stack[maxn], top=-1;  
    for(int i=0; i<n; i++){ //计算right[i]  
        while(top>=0 && h[i]<h[stack[top]]){  
            int k = stack[top--];  
            right[k]=i;  
        }  
        stack[++top] = i;  
    }  
}
```



维护单调递增栈

```
void PreviousSmaller(int h[], int left[], int n) {  
    //计算left[i], 即h[i]左边第1个比h[i]小的元素位置  
    for(int i=0; i<n; i++) left[i]=-1;  
    int stack[maxn], top=-1;  
    for(int i=n-1; i>=0; i--){ //反向扫描, 计算left[i]  
        while(top>=0 && h[i]<h[stack[top]]){  
            int k = stack[top--];  
            left[k]=i;  
        }  
        stack[++top] = i;  
    }  
}
```



柱状图中最大矩形

```
int LargestRectangle(int h[], int n) {  
    int right[maxn], left[maxn];  
    NextSmaller(h, right, n);  
    PreviousSmaller(h, left, n);  
    int MaxRect=-1;  
    for(int i=0; i<n; i++){ //考察每个柱子  
        int CurRect = (right[i]-left[i]-1)*h[i];  
        if(CurRect > MaxRect)  
            MaxRect = CurRect;  
    }  
    return MaxRect;  
}
```

时间复杂度 $O(n)$



滑动窗口最大值

给定一个长度不超过 10^5 的整数数组 a ，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。滑动窗口每次只向右移动一位。返回滑动窗口中的最大值。【腾讯、华为、字节跳动、拼多多、阿里、百度、快手、微软、谷歌、苹果面试题 [LeetCode2359](#)】

例如：

输入：[1, 3, -1, -3, 5, 3, 6, 7], $k = 3$

输出：[3, 3, 5, 5, 6, 7]

输入: $[1, 3, -1, -3, 5, 3, 6, 7]$, $k = 3$

输出: $[3, 3, 5, 5, 6, 7]$

滑动窗口的位置							最大值	
[1	3	-1]	-3	5	3	6	7	3
1	[3	-1	-3]	5	3	6	7	3
1	3	[-1	-3	5]	3	6	7	5
1	3	-1	[-3	5	3]	6	7	5
1	3	-1	-3	[5	3	6]	7	6
1	3	-1	-3	5	[3	6	7]	7



暴力方法

```
for(int i=0; i<n; i++){ //求窗口区间[i,i+k)的最大值
    for(int j=i; j<i+k; j++){
        }
    }
```

时间复杂度 $O(nk)$

滑动窗口的位置

最大值

[9	2	3	7]	5	6	8	3	7	9
9	[2	3	7	5]	6	8	3	7	7
9	2	[3	7	5	6]	8	3	7	7
9	2	3	[7	5	6	8]	3	7	8



滑动窗口的位置

最大值

[9 2 3 7] 5 6 8 3 7

9

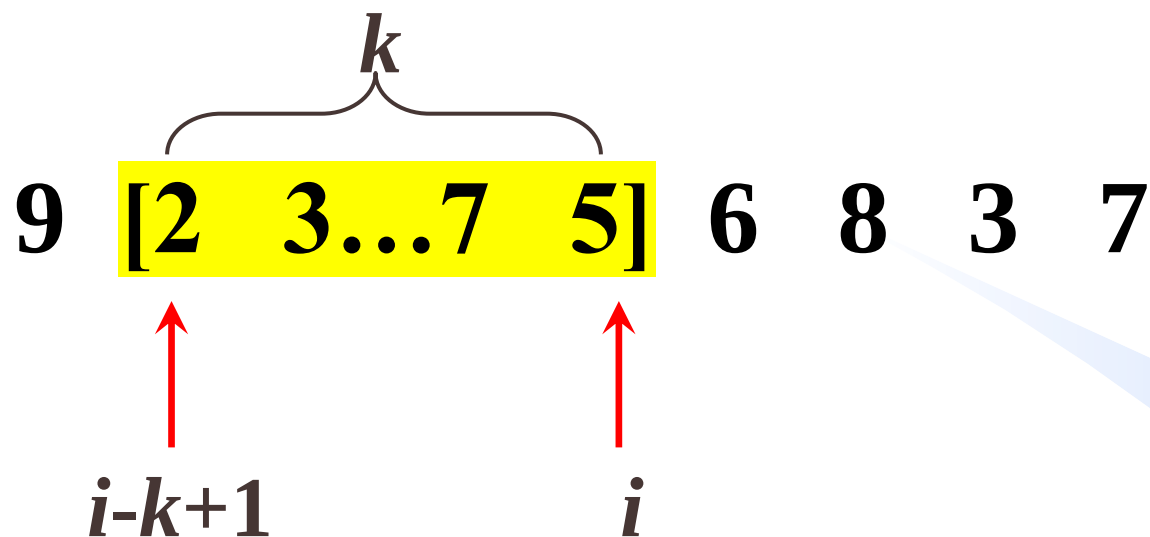
单调队列



```
const int maxn = 1e5 + 10;
void SlidingWindowMax(int a[], int n, int k){
    int queue[maxn], left=0, right=0;
    for(int i=0; i<k; i++){ //依次扫描第1个窗口的各元素
        while(left<right && a[i]>=a[queue[right-1]])
            right--;
        queue[right++] = i;
    }
    printf("%d ", a[queue[left]]); //左侧队头为第1个窗口的最大值
    //继续扫描后续窗口
    for (int i=k; i<n; i++) {
        while(left<right && a[i]>=a[queue[right-1]])
            right--;
        queue[right++] = i;
        //此时左侧队头还是窗口最大值么？取决于队头是否还在窗口里
    }
}
```

单调递减队列

[9 2 3 7] 5 6 8 3 7



若左侧队头的下标 $< i-k+1$ ，则说明
着其不在窗口里，应将其出队



```
const int maxn = 1e5 + 10;
void SlidingWindowMax(int a[], int n, int k){
    int queue[maxn], left=0, right=0;
    for(int i=0; i<k; i++){ //依次扫描第1个窗口的各元素
        while(left<right && a[i]>=a[queue[right-1]])
            right--;
        queue[right++] = i;
    }
    printf("%d ", a[queue[left]]); //左侧队头为第1个窗口的最大值
    //继续扫描后续窗口
    for(int i=k; i<n; i++) {
        while(left<right && a[i]>=a[queue[right-1]])
            right--;
        queue[right++] = i;
        if(queue[left]<i-k+1) left++; //若队头不在窗口里，则出队
        printf("%d ", a[queue[left]]);
    }
}
```

单调递减队列

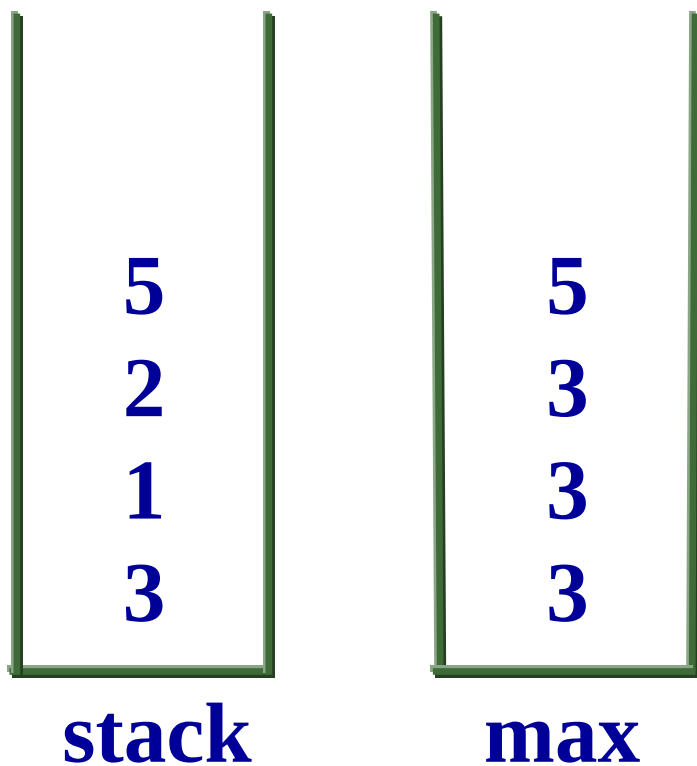
时间复杂度 $O(n)$

栈最大元素

- 实现一个特殊的栈，在基本功能基础上，增加一个“返回栈中最大元素”的操作 **GetMax**，要求该操作时间复杂度 $O(1)$ 。

PUSH(x)

- ① $\text{stack} \leftarrow x$
- ② 如果 **max** 栈空：则 $\text{max} \leftarrow x$
 如果 **max** 栈不空：
 - ✓ x 大于 **max** 栈顶元素： $\text{max} \leftarrow x$
 - ✓ x 小于等于 **max** 栈顶元素： **max** 栈顶元素重复压入 **max**



GetMax(): 取 **max** 栈顶元素

POP(): **stack** 和 **max** 均弹栈



最大频率栈

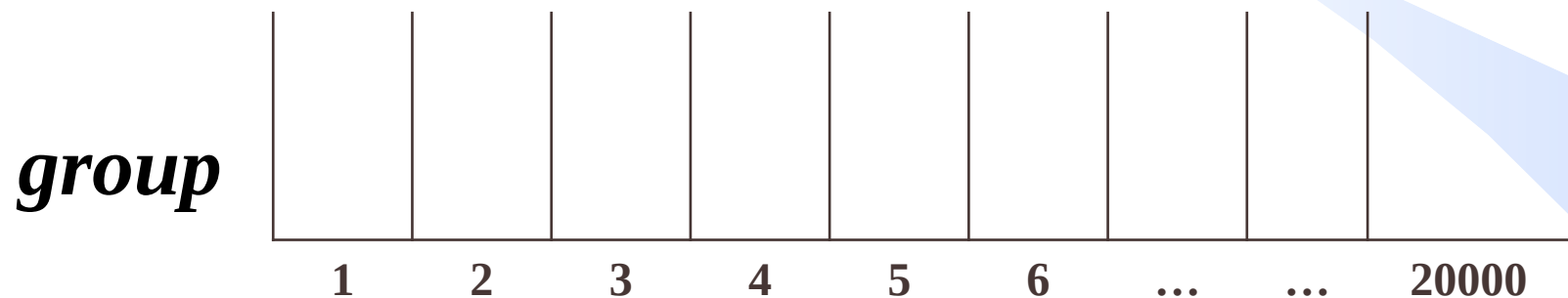
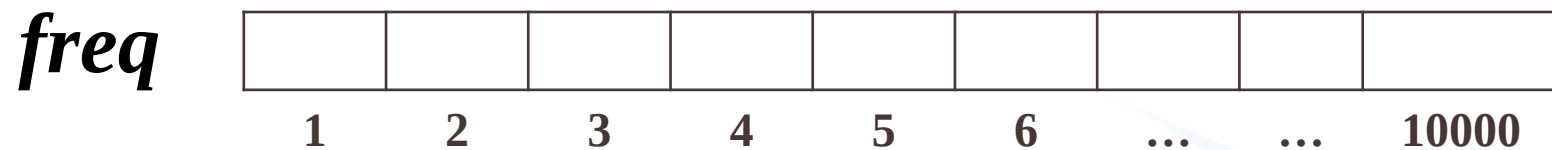
实现类似栈的数据结构FreqStack。FreqStack 有两个函数：

- ✓ `push(int x)`: 将整数 x ($1 \leq x \leq 10^4$) 压入栈中。假设压栈次数不超过 2×10^4 次。
- ✓ `pop()`: 它移除并返回栈中出现最频繁的元素。如果最频繁的元素不只一个，则移除并返回最接近栈顶的元素。

【阿里面试题】

示例：

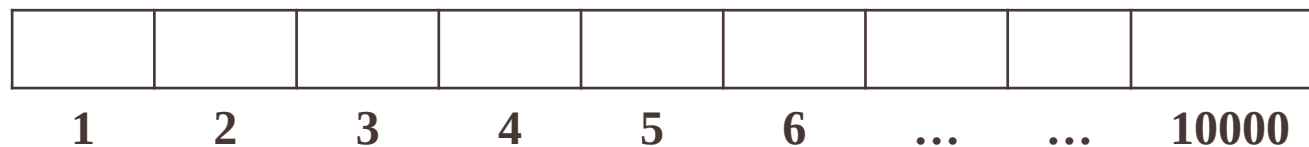
- **push [5, 7, 5, 7, 4, 5]**
- **pop():** 返回5，因为5是频率最高的。栈变成 [5, 7, 5, 7, 4]。
- **pop():** 返回 7，因为 5 和 7 都是频率最高的，但 7 最接近栈顶。栈变成 [5, 7, 5, 4]。
- **pop():** 返回 5 。 栈变成 [5, 7, 4]。
- **pop():** 返回 4 。 栈变成 [5, 7]。



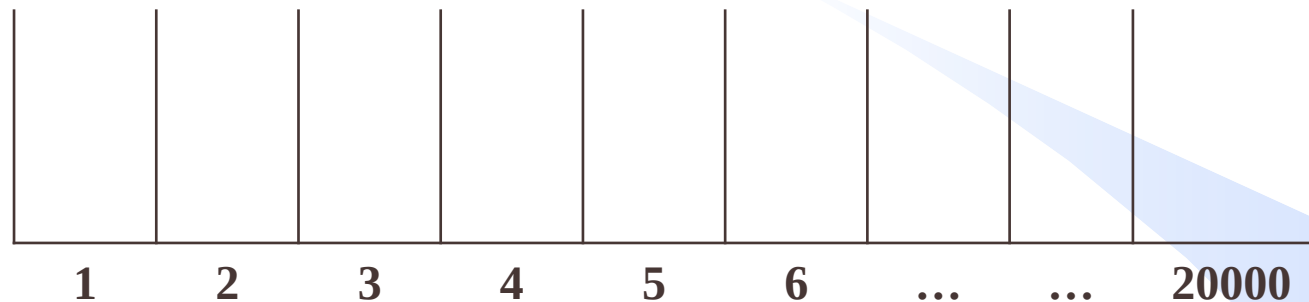
$\text{freq}[x]$: 元素 x 出现的次数

$\text{group}[f]$: 所有频率为 f 的元素（按进栈顺序形成一个栈）

freq



group



时间复杂度 $O(1)$

压栈:

```
freq[x]++;  
int f=freq[x];  
if(f>maxfreq)  
    maxfreq=f;  
group[f].push(x);
```

弹栈:

```
int x=group[maxfreq].pop();  
freq[x]--;  
if(group[maxfreq].empty())  
    maxfreq--;  
return x;
```




自愿性质OJ练习题

✓ 洛谷P2947

✓ 洛谷P1886